

DD2350 Algoritmer, datastrukturer och komplexitet

Övning 8

Oavgörbarhet – var gränsen för algoritmer går

KTH · HT 2026

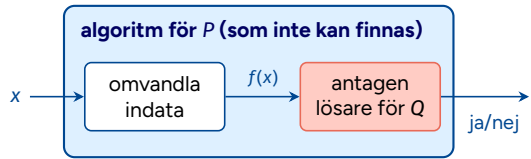
I dag

Först en kort repetition. Sedan, för varje uppgift: frågan på en bild och lösningen på nästa.

1. **Orm i kakel** – tre varianter.
2. **Ett program** där stopp är avgörbart?
3. **Ett program** där stopp är oavgörbart?
4. **MR** – den maximala körtiden.
5. **MR** mot rekursiva funktioner.
6. **En turingmaskin** på högst *K* rutor.
7. **Rekursiva mängder** och rekursivt uppräkneliga.
8. **Den diagonaliserade stoppmängden.**

Repetition: avgörbart, oavgörbart, reduktion

- Ett beslutsproblem är **avgörbart** om det finns en algoritm som alltid stannar och svarar rätt ja eller nej.
- För att visa att Q är **oavgörbart**: reducera ett känt oavgörbart problem P till Q. Gick Q att avgöra, gick P också.
- Reduktionen måste själv vara en algoritm som alltid stannar. Den får anropa lösaren för Q flera gånger.
- Kända oavgörbara problem: stopproblemet, stopp på tomt indata, orm i kakel.



Riktningen

"P reduceras till Q" visar att Q är minst lika svårt som P. Att reducera Q till något oavgörbart visar ingenting om Q.

Uppgift 1: orm i kakel

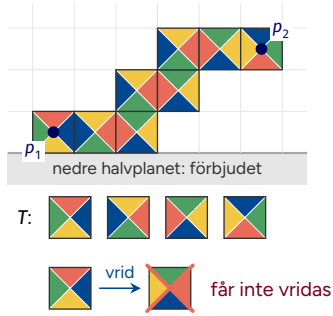
Problemet **orm i kakel** tar som indata en mängd T av kakelplattstyper och två punkter p_1 och p_2 i planet. Frågan är om det går att, med bara kakeltyper i T, kakla en orm som ringlar sig från p_1 till p_2, som inte bryter kakelmönstret någonstans och som hela tiden håller sig i det övre halvplanet. **Plattorna får inte vridas.**

Är följande varianter avgörbara eller oavgörbara?

a) Indata utvidgas med en kakeltyp $t \in T$. Första plattan, den som täcker p_1, måste vara av typ t.

b) Indata utvidgas med ett tal N. Ormen får bestå av högst N plattor.

c) Indata utvidgas med ett tal N. Ormen måste bestå av minst N plattor.



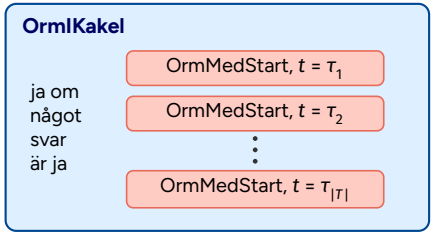
Från föreläsningen: orm i kakel är oavgörbart.

Lösning 1a: oavgörbart

Reducera orm i kakel **till** varianten:

```
OrmIKakel(T, p1, p2) =
1   for each t ∈ T do
2       if OrmMedStart(T, p1, p2, t)
            then return true
3   return false
```

En orm finns om och endast om den finns med någon första platta $t \in T$. T är ändlig, så slingan stannar.



Slutsatsen

Gick OrmMedStart att avgöra skulle orm i kakel också gå. Det gör det inte, alltså är varianten oavgörbar.

Lösning 1b: avgörbart

Pröva alla ormar med högst N plattor som börjar i p_1:

```
OrmHögstN(T, p_1, p_2, N) =
1   for each orm med högst N plattor från p_1 do
2       if den följer mönstret, håller sig i det övre
            halvplanet och slutar i p_2 then return true
3   return false
```

Varje ny platta ligger bredvid den förra – 4 riktningar för den andra, sedan högst 3 – och har en av $|T|$ typer. Det ger högst $(3|T|)^N$ ormar. Ändligt många, så slingan stannar.



Avgörbart är inte effektivt

$(3|T|)^N$ är exponentiellt i N. Det räcker att det är ändligt.

Om plattorna fick vridas

Då vore det $4|T|$ val per platta i stället för $|T|$ – fortfarande ändligt.

Lösning 1c: oavgörbart

Reducera orm i kakel **till** varianten:

```
OrmIKakel(T, p1, p2) =
1     return OrmMinstN(T, p1, p2, 1)
```

Varje orm har minst en platta, så frågorna har samma svar. Gick OrmMinstN att avgöra skulle orm i kakel också gå.

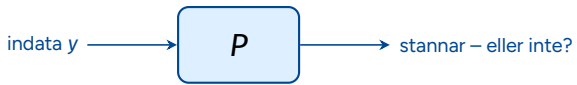
En undre gräns stänger inget

Högst N gör sökrummet ändligt. Minst N lämnar det lika oändligt som förut.

Kuriosa: i hela planet är ormproblemet avgörbart (PSPACE-fullständigt). Det är halvplanet som gör det oavgörbart.

Uppgift 2: avgörbart stopp för ett *P*?

Finns det något explicit program *P* så att det, givet *y*, är **avgörbart** huruvida *P* stannar på indata *y*?



P är fixt. Bara *y* varierar.

Lösning 2: ja, massor

```
   P(y) =
1      return
```

P stannar på alla indata, så algoritmen ”svara ja” avgör frågan – utan att köra *P* alls.

```
   P_2(y) =
1      while y är udda do ingenting
2      return
```

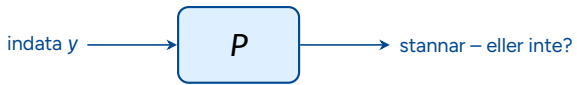
P_2 stannar precis när *y* är jämnt: ”svara ja om *y* är jämnt”. Ett program som aldrig stannar fungerar också: ”svara nej”.

Avgörbart betyder

att det finns en algoritm som alltid stannar och svarar rätt. Den behöver inte simulera *P*: här räcker det att läsa programmet.

Uppgift 3: oavgörbart stopp för ett *P*?

Finns det något explicit program *P* så att det, givet *y*, är **oavgörbart** huruvida *P* stannar på indata *y*?

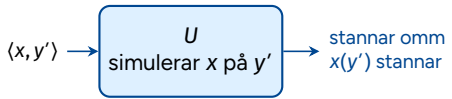


P är fixt. Bara *y* varierar.

Lösning 3: ja, en interpretator

Låt P vara en interpretator U. Indata är ett par $y = \langle x, y' \rangle$: ett program x och ett indata y' till det. $U(y)$ gör precis det x skulle göra på y'.

Att avgöra om U stannar på y är då att avgöra om x stannar på y'. Det är stopproblemet, och det är oavgörbart.



Oavgörbart handlar om problem

Ett problem har oändligt många instanser. För ett fixt P kan frågan vara trivial eller omöjlig, beroende på vilket P det är.

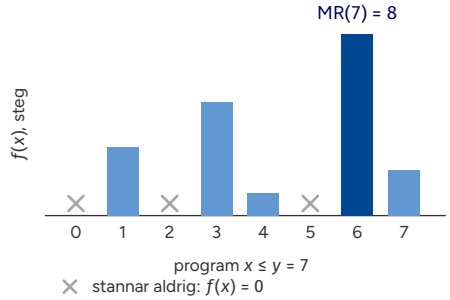
Uppgift 4: MR, den maximala körtiden

Om programmet x stannar på tomt indata låter vi $f(x)$ vara antalet steg innan det stannar. Annars sätter vi $f(x) = 0$.

Definiera MR(y), den maximala körtiden över alla program vars binära kodning är mindre än y:

$$\mathrm{MR}(y) = \max_{x \le y} f(x).$$

Är MR beräkningsbar?

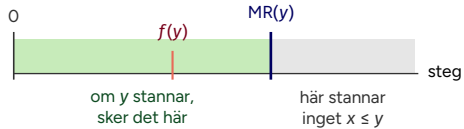


Lösning 4: MR är inte beräkningsbar

Om MR vore beräkningsbar kunde vi avgöra stopp på tomt indata:

```
StoppTomt(y) =
1    s ← MR(y)
2    simulera y på tomt indata i högst s steg
3    return true om y stannade, annars false
```

Stopp på tomt indata är oavgörbart. Alltså kan MR inte vara beräkningsbar.



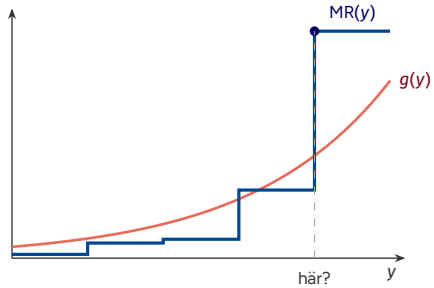
Varför det fungerar

$y \leq y$, så $f(y) \leq \text{MR}(y)$. Har y inte stannat efter $\text{MR}(y)$ steg, stannar det aldrig.

Uppgift 5: MR mot rekursiva funktioner

Visa att funktionen MR i föregående uppgift växer snabbare än varje rekursiv funktion.

Visa närmare bestämt att för varje rekursiv funktion g finns ett y så att $\mathrm{MR}(y) > g(y)$.



Lösning 5: en motsägelse

Anta motsatsen: det finns en rekursiv g med $g(y) \geq \mathrm{MR}(y)$ för alla y.

Byt $s \leftarrow \mathrm{MR}(y)$ mot $s \leftarrow g(y)$ i StoppTomt från uppgift 4. Programmet svarar fortfarande rätt – det simulerar bara längre – och nu är det beräkningsbart. Motsägelse.

Alltså: för varje rekursiv g finns ett y med $\mathrm{MR}(y) > g(y)$.

Till och med oändligt många: vore de ändligt många kunde vi rätta g på just de ställena med en tabell.

Släkt med Busy Beaver

Radós $S(n)$: det största antalet steg som en stannande turingmaskin med n tillstånd och två symboler kan ta på tomt band. $S(5) = 47\,176\,870$ bevisades först 2024.

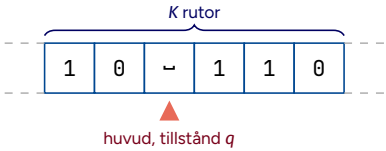
Inte samma sak som ”snabbt växande”

2^{2^y} växer fort men är beräkningsbar. MR växer fortare än *varje* sådan funktion.

Uppgift 6: högst K rutor på bandet

Anta att en turingmaskin M, indata X (som står på bandet från början) och en heltalskonstant K är givna. Är följande problem avgörbart eller oavgörbart?

Stannar M på indata X efter att ha använt högst K rutor på bandet? En använd ruta får skrivas och läsas flera gånger.



Lösning 6: avgörbart

En **konfiguration** är tillstånd, huvudets position och bandets innehåll. Så länge M håller sig inom K rutor finns högst

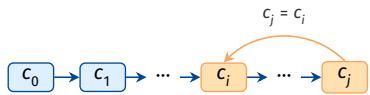
$$C = m \cdot K \cdot 3^K$$

konfigurationer: m tillstånd, K positioner och 3 symboler per ruta.

M är deterministisk. Samma konfiguration ger alltid samma nästa steg.

Simulera $C + 1$ steg:

- M stannar: **ja**.
- M lämnar de K rutorna: **nej**.
- Annars har en konfiguration upprepats (lådprincipen), och M går runt i en evig slinga: **nej**.



Uppgift 7: rekursiva mängder

En rekursivt uppräkningsbar mängd definierades på föreläsningen som ett språk som kan kännas igen av en funktion vars ja-certifikat kan verifieras ändligt.

En alternativ definition är en mängd som är uppräknelig (elementen kan numreras med naturliga tal) och kan produceras av en algoritm.

Använd den senare definitionen och visa att varje rekursiv mängd också är rekursivt uppräknelig.

Rekursiv mängd S

Det finns en algoritm $A(x)$ som alltid stannar och svarar om $x \in S$.

Den senare definitionen

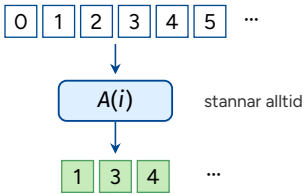
S är rekursivt uppräknelig om en algoritm skriver ut elementen i S, ett i taget och kanske utan slut.

Lösning 7: en filtrerande slinga

Elementen i S är som allt annat binära strängar, alltså tal. Låt A avgöra S och skriv ut det den säger ja till:

```
1 for i ← 0 to ∞ do
2     if A(i) then write i
```

Varje $x \in S$ skrivs ut när $i = x$, och inget annat skrivs ut. Det fungerar för att A alltid stannar.



Om A bara kan säga ja

Stannar A inte för något $i \notin S$ fastnar slingan där. Då behövs knepet i uppgift 8.

Uppgift 8: stoppmängden

Den diagonaliserade stoppmängden består av alla program p som stannar på indata p.

Visa att denna mängd är rekursivt uppräknelig.



Att fundera på

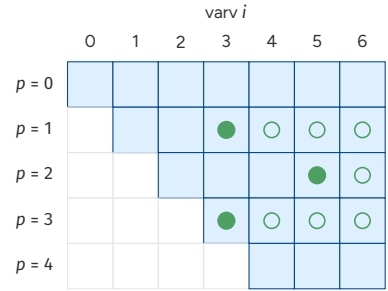
Varför går det inte att göra som i uppgift 7?

Lösning 8: simulera i diagonaler

Slingan från uppgift 7 fastnar: $p(p)$ kanske aldrig slutar, och då kommer vi aldrig vidare till nästa p. Simulera i stället i diagonaler:

```
1  for i ← 0 to ∞ do
2      for p ← 0 to i do
3          simulera p(p) i högst i steg
4          if p(p) stannade then write p
```

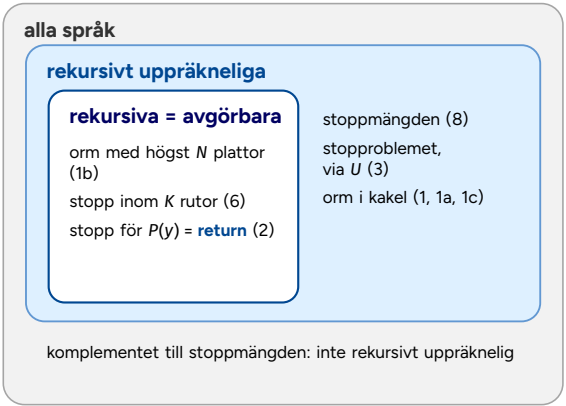
Stannar $p(p)$ efter s steg skrivs p ut i varje varv $i \geq \max(p, s)$. Inget program som aldrig stannar skrivs ut.



Fylld cirkel: första utskriften. Ring: samma p igen. Program 0 och 4 stannar aldrig.

Varje p en gång: skriv ut p bara om $p(p)$ stannar på exakt i steg, eller om $p = i$ och det stannar inom i steg.

Var gränsen går



- **Ändligt sökrum** ger avgörbart, även när det är dyrt (1b, 6).
- **Obegränsat sökrum:** reducera från ett känt oavgörbart problem (1a, 1c, 4).
- **Ett enskilt program** kan vara lätt eller omöjligt (2, 3).
- **Avgörbar** ger rekursivt uppräknelig (7). Stoppmängden är rekursivt uppräknelig men inte avgörbar (8).

Innehåll, som avslutning

1. **Orm i kakel** – a och c oavgörbara genom reduktion, b avgörbart: ändligt många ormar.
2. **Avgörbart stopp** – ja: $P(y)$ = **return**.
3. **Oavgörbart stopp** – ja: en interpretator.
4. **MR** – inte beräkningsbar, för då kunde vi avgöra stopp på tomt indata.
5. **MR mot rekursiva funktioner** – ingen rekursiv g ligger över MR överallt.
6. **Högst** K **rutor** – ändligt många konfigurationer: avgörbart.
7. **Rekursiv mängd** – filtrera $0, 1, 2, \ldots$ genom A.
8. **Diagonaliserade stoppmängden** – rekursivt uppräknelig: simulera i diagonaler.



Frågor? Lösningarna finns också i övningshäftet.